

**Paraphrase Detection
Working model**

**BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING**

Submitted By

Dennis Mathuram	222010301028
M Anirudh Reddy	222010301026
Raj Nakshatra Sharma	222010301024
K Divyanand	222010301014

Under The Guidance Of

Dr B Krishnaveni

Assistant Professor



Department Of Computer Science and Engineering GITAM School of Technology
GITAM (Deemed to be University)

Hyderabad-502329
September, 2023

Table Of Contents

ABSTRACT	3
INTRODUCTION	3
PARAPHRASE TYPES.....	5
LITERATURE SURVEY	7
PROBLEM ANALYSIS	8
DESIGN.....	10
NATURAL LANGUAGE PROCESSING (NLP).....	10
DATASET	13
IMPLEMENTATION	15
SYSTEM REQUIREMENTS.....	15
BERT MODEL	16
TF-IDF VECTORIZERS.....	16
USING STREAMLIT.....	17
FEATURE EXTRACTION AND DATA PREPROCESSING.....	19
TESTING AND VALIDATION	22
RESULT	35
CONCLUSION	37
REFERENCES.....	38

Abstract

Paraphrase detection is important for several applications, including plagiarism detection, authorship attribution, question answering, text summarization, text mining in general, etc. To do this project we have used various models and Quora-Question-Pair dataset.

The objective of this research is to detect matching segments of text as paraphrase types from a pair of source and paraphrased sentences. We also aim to identify whether an extracted text segment (as a paraphrase type) is a synonymous substitution or a word reordering.

Here we have built a Streamlit web application for paraphrase detection using a pre-trained model. This project is a part of a Natural Language Processing (NLP) project that aims to build a model for comparing and measuring the similarity between two text-based questions or sentences. The code performs a series of preprocessing, feature extraction, and data transformation steps to create a set of features for comparing two questions.

Introduction

Paraphrase Identification is a task where a model should identify whether a pair of sentences or documents is a paraphrase. In a business context, paraphrase identification is significantly helpful to improve the user experience of the platform. For instance, in a question-answer forum such as Quora, the user may have the same question that has been asked and answered. Therefore, before the user creates a post, there is a pop-up suggestion for a similar question. Furthermore, paraphrase identification is also used in many natural language processing applications such as machine translation, information retrieval, answer selection question and answering system, and more.

We also found numerous recent works on semantic text similarity. The goal of the task is the same as paraphrase identification, to decide how identical of two pairs of sentences. However, the approach is different from paraphrase identification. Text similarity relies on similar metrics such as Cosine similarity and other various similarity metrics and the result ranges from 0 to 1. Therefore, to decide similar or not you have to set a threshold for the score (Fernando and Stevenson, 2008; Neves et al., 2019; Kim et al., 2019; He et al., 2015; Hussain and Suryani, 2015)

In this study, we examine Quora Question Pairs dataset using Bag of Words (BoW) and Word Piece to chunk the text and using tree boosting algorithm that already widely used such as Catboost, XGBoost. Moreover, we also conducted a study on deep learning algorithms such as LSTM and BERT to see how effective it is to classify paraphrase identification.

Natural language processing has been the focus of technological contributions recently. This is evident in many applications such as extracting information from big data on the web although most of those data are in unstructured format. Much has been accomplished in studying sentiment analysis in social networks and in the field of grammatical analysis that is used heavily in essay marking and assessing student's answers. Paraphrase detection (PD) has always been problematic in the study of natural language processing (NLP). The goal is to establish if a pair of sentences is conveying identical meaning or not. Two sentences are judged as paraphrases if they yield identical meaning. If the meaning is not identical, they are considered non-paraphrased. Most of the existing systems such as Microsoft paraphrase corpus (MSRP) are efficient at cleaning text

corpora. (PD) is deemed a fundamental subtask in many NLP tasks, it obviously appears in for example question answering, finding similar relations between question needs paraphrases, also it's widely used in machine translation, document clustering and Information retrieval, etc. The survey at hand briefly investigates the current research done on PD by means of presenting them in two approaches. The first approach is an unsupervised approach that heavily depends on text similarity. The second is the supervised approach that depends on machine learning (ML) and deep learning (DL).

Paraphrase detection in user-generated noisy texts, such as Twitter texts,¹ is an important task for various Natural Language Processing (NLP), information retrieval and text mining tasks, including query ranking, plagiarism detection, question answering, and document summarization. Recently, the paraphrase detection task has gained significant interest in applied NLP because of the need to deal with the pervasive problem of linguistic variation.

The paper uses GloVe as a pretrained word vector from 840B Common Crawl corpus and applies it to Quora Question Pairs and selects random sample from it. The paper also benchmarks with other approaches, and the proposed model, BiMPM gives a stunning result that achieves 88.8 percent accuracy.

Paraphrase detection is an NLP classification problem. Given a pair of sentences, the system determines the semantic similarity between the two sentences. If the two sentences convey the same meaning, then it is labeled as paraphrase; otherwise, it is labeled as non-paraphrase. Most of the existing paraphrase systems have performed quite well on clean text corpora, such as the Microsoft Paraphrase Corpus (MSRP) (Dolan, Quirk, & Brockett, 2004). However, detecting paraphrases in user-generated noisy Tweets is more challenging due to issues like misspelling, acronyms, style, and structure (Xu, Ritter, Callison-Burch, Dolan, & Ji, 2014). In addition, measuring the semantic similarity between two short sentences is very difficult due to the lack of common lexical features (Kajiwara, Bollegala, Yoshida, & Kawarabayashi, 2017). Although little attention has been given to paraphrase detection in noisy short-texts thus far, some initial work has been reported on the SemEval 2015 benchmark Twitter dataset (Dey, Shrivastava, Kaushik, 2016, Xu, Callison-Burch, Dolan, 2015, Xu, Ritter, Callison-Burch, Dolan, Ji, 2014). Unfortunately, the best performing approaches on one dataset don't seem to perform as good when evaluated against another. As we discuss later in this paper, the state-of-the-art approach for the SemEval dataset proposed by Dey et al. (2016) does not have good performance (in form of F1-score) when evaluated on the MSRP dataset. Similarly, Ji and Eisenstein (2013) are the best performing approach on the MSRP dataset, but do not perform well on the SemEval dataset. In conclusion, existing approaches are not very generic, but rather are highly dependent on the data used for training.

Paraphrase identification is beneficial for several significant natural language processing (NLP) applications. For instance, a summarization system can eliminate redundancy when generating summaries by removing paraphrases. Additionally, paraphrases can be used to improve the performance of question-answering systems.

Perhaps most notably, paraphrase identification is essential for detecting plagiarism. In the context of intelligent tutoring systems, paraphrase identification can be used to evaluate whether a student's submission or answer is semantically equivalent to a reference answer. Overall, paraphrase identification has a variety of applications for assessing the quality of natural language text.

Paraphrase Types

1. **Same Polarity Substitutions:** Same polarity substitutions, also known as synonym substitutions, involve the replacement of a word or phrase of the sentence with one of its synonyms. Same polarity substitutions have three. Subtypes: 1.

- a. habitual (replacing a verb with its synonym),
- b. contextual (replacing a phrase with an equivalent phrase based on context), and
- c. named entities (replacing a noun with its equivalent alternative).

Example: I dislike doing extra work. = I hate doing extra work.

2. **Opposite Polarity Substitutions:** Opposite Polarity Substitutions are also known as antonym substitutions, where words and phrases in a sentence are replaced by their negated antonyms. There are two types of substitutions.

- a. habitual (main verb or adverbs are replaced by one of its antonyms) and
- b. contextual (phrases are replaced by their alternative with opposite meanings)

Example: The program runs fast = The program does not run slowly

3. **Converse Substitution:** Converse substitution involves the relational substitution of a word in a sentence by its relational pair with an opposite viewpoint.

Example: I bought a plane ticket online = A plane ticket was sold to me online

4. **Inflectional Changes:** Inflectional changes involve the inflection of nouns (usually inflected for numbers) and the inflection of verbs (inflected for tense).

Example: Increase in salaries is often a great indicator of performance = Increase in salary is often a great indicator of performance.

5. **Sentence Modality Changes:** Sentence modality changes involve the overall change in the expression of perspectives regarding certainty toward the subject of the sentence.

Example: Does working in that technology company pay well? Does it provide great 401k plans for its employees? = They will work at the company to get high pay or to obtain great 401k plans.

6. **Functional Word Substitution:** Functional word substitution involves substituting a functional word in a sentence with another functional word.

Example: Is this your own work? = Is that your own work?

- 7. Spelling Changes:** Spelling changes involve changes through contractions (combining two words) or verb conjugations.

Example: The countless hours spent practicing did not improve our performance = The countless hours spent practicing didn't improve our performance.

- 8. Structure/Discourse Changes:** Structure and discourse changes involve changes in the referencing context of the discourse in a sentence.

Example: How he would stare! = He would surely stare!

- 9. Relational Substitutions:** Relational substitutions involve the substitution of a word or phrase by its relational counterparts. The relational counterparts are defined in two ways:

- a. agent(action)/action(agent) substitution,
- b. manipulator/device substitution.

Example: Jacob programmed the game = the game's programmer is Jacob

- 10. Direct/Indirect Style Alternation:** Direct and indirect style alternation involves the changing of voices in a sentence such as from active to passive voice and vice versa.

Example: "You must finish the project by the end of today," demanded my manager = My manager demanded that I must finish the project by the end of today.

Literature Survey

A New Online Plagiarism Detection System based on Deep Learning paper proposed an online plagiarism detection system which is based on Doc2vec technique for word embedding in coordination with SLSTM and CNN deep learning algorithms. “Code Plagiarism Detection Method Based on Code Similarity and Student Behavior Characteristics paper proposed the concept of code similarity concentration. We learn to focus on the similarity between two documents from this paper. “Machine Learning Models for Paraphrase Identification and its Applications on Plagiarism Detection” paper identify if a sentence is a paraphrase of another one. Paraphrasing is where you copy someone’s exact words and put them in quotation marks. From this paper we studied LSTM and RNN algorithms. Plagiarism detection is done using string matching algorithm, k-gram and karp Rabin algorithm. The algorithms are used to detect the originality of students work based on similarity concentration. “Plagiarism Detection Using Machine Learning-Based Paraphrase Recognizer” paper studied a support vector machine-based paraphrase recognition system. SVM is one of the algorithms in machine learning, which works by extracting lexical, syntactic, and semantic features from.

In this Project, Python Framework and Deep Learning Will be used. Python will be used in frontend as well as in backend. Python will also be used to implement deep learning algorithms, which will help in using Doc2vec module for data preprocessing. Also help in building CNN architecture. First, we will take paragraph or suspicious document for plagiarism checking from user. Line by line all the sentences from the document will be taken and passed to Doc2vec where data pre-processing will be done. Later on, the sentence vectors which are given as an output after data preprocessing will be shared with first learning phase which is based on LSTM algorithm and then from the CNN algorithm for plagiarism detection.

Problem Analysis

Paraphrase detection, the task of identifying whether two text segments convey the same meaning despite differences in wording, is a critical component of natural language processing with a wide range of applications, including information retrieval, question answering, and text summarization. Traditional approaches to paraphrase detection often relied on linguistic and lexical features and manually crafted rules. However, these approaches have limitations in handling the nuances of natural language, especially in cases where paraphrases exhibit variations in semantics and language usage.

These models are expected to outperform traditional methods by automatically learning hierarchical representations of text, capturing both local and long-range dependencies, and handling the complexity of natural language, including variations in sentence structure, writing style, and language.

Several challenges and limitations surround this problem:

Data Availability and Quality: Acquiring a sufficiently large and high-quality dataset of paraphrase and non-paraphrase sentence pairs for training deep-learning models is challenging. The dataset needs to be diverse, cover multiple domains and writing styles, and be accurately annotated.

Semantic Variability: Paraphrases exhibit a wide range of semantic variability, making it challenging for deep-learning models to capture the subtle differences in meaning between sentences. The models need to be able to differentiate between near-synonyms, antonyms, and various forms of reformulations.

Negation Handling: Detecting and handling negations and double negations that reverse the meaning of sentences is a complex task for deep-learning models. Effective negation handling is crucial for accurate paraphrase detection.

Out-of-Vocabulary Words: Deep-learning models often rely on pre-trained word embeddings. Handling out-of-vocabulary (OOV) words, which are not present in the embeddings, is a challenge, as OOV words can carry important semantic information.

Long Dependencies: RNN-based models struggle with capturing long-range dependencies in text, which are crucial for accurate paraphrase detection. Handling long sentences or documents that span multiple paragraphs or pages is essential.

Cross-Lingual and Cross-Domain Challenges: Adapting deep-learning models for paraphrase detection in different languages and domains introduces complexities due to variations in sentence structures, word orders, and domain-specific terminology.

Stylistic Variations: Paraphrases in text with varying writing styles, including formal vs. informal language or technical vs. non-technical language, pose challenges. Deep-learning models need to be robust to these variations.

Interpretable Models: While deep-learning models often achieve high accuracy, they are often perceived as "black boxes." Ensuring that these models are interpretable is important, particularly in contexts where decisions have critical consequences.

Ethical and Bias Considerations: Addressing ethical considerations, such as bias in data and model outputs, is essential in paraphrase detection. Models should be designed to mitigate biases and produce ethically sound results.

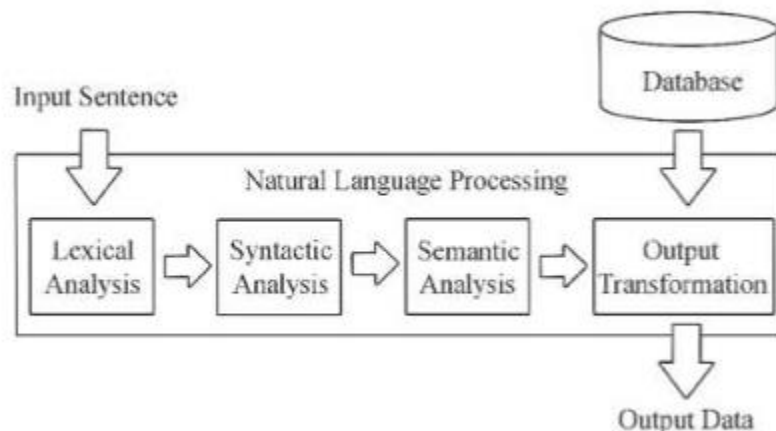
Adversarial Attacks: Deep-learning models are susceptible to adversarial attacks, where malicious inputs can mislead the model. Ensuring model robustness against such attacks is a key challenge.

Design

Natural Language Processing (NLP) is a subfield of linguistics, computer science, information engineering, and artificial intelligence concerned with the interactions between computers and human (natural) languages, how to program computers to process and analyze large amounts of natural language data. Challenges in NLP often refer to natural language understanding, speech recognition, and natural language generation.

Lexical Analysis (also known as scanning or tokenization):

Lexical analysis is the first phase of compiling or interpreting a programming language. It involves breaking the source code into a sequence of lexemes (or tokens). Lexemes are the smallest meaningful units in a programming language, such as keywords, identifiers, literals, and operators. The purpose of lexical analysis is to identify and categorize these lexemes, stripping away white spaces and comments, to provide the syntactic analyzer (parser) with a stream of tokens that can be processed further. Lexical analyzers are responsible for recognizing and generating tokens based on the language's syntax rules.



- **Tokenization** -Tokenization is, generally, an early step in the NLP process, a step which splits longer strings of text into smaller pieces, or tokens. Larger chunks of text can be tokenized into sentences, sentences can be tokenized into words, etc. Further processing is usually achieved after a piece of text has been properly tokenized.
- **Normalization** - Before processing ahead, text must be normalized. Normalization usually refers to a sequence of related tasks involved to put all text on a level playing field: converting all text to the same case (upper or lower), eliminating punctuation, increasing contractions, converting numbers to their word identification, and so on. Normalization brings all words on the same footing and allows processing to continue consistently.
- **Stemming** - Stemming is the process of discarding affixes from a word to gain a word stem.

- **Corpus** -In linguistics and NLP, corpus refers to a collection of texts. Such collections may be built on a single language of texts or can span different languages -- there are various reasons for which multilingual corpora may be useful. Corpora may also subsist on the media texts. Corpora are frequently individually used for statistical linguistic analysis and hypothesis testing.

- **Stop Words** - Stop words are those words which are clean out of text, since these words share limited to, overall meanings, given that they are usually the most familiar words in a language. For example, "the," "and" and "a," while all needed words in an appropriate passage, don't usually contribute greatly to one's understanding of content.

- **Parts-of-speech (POS) Tagging**- POS tagging subsists of appointing a category tag to the tokenized parts of a sentence. The most famous POS tagging would be classified words as nouns, verbs, adjectives, etc.

- **Bag of Words**- Bag of words is a special representative model used to clarify the details of a selection of text. The bag of words model discards grammar and word order but is attentive in the number of existence of words within the text. The entire illustration of the document selection is that of a bag of words.

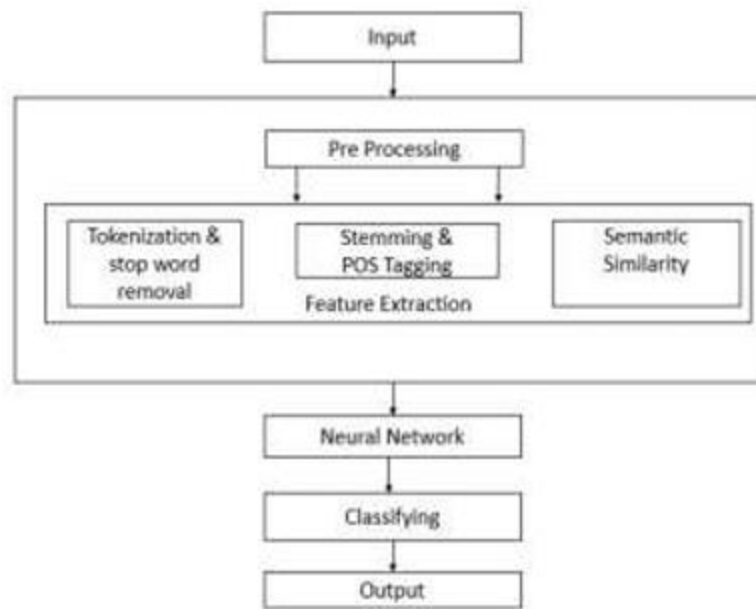
There are various similarity measures which can be enforced to NLP. What are we measuring the similarity of? Mostly, strings.

- **Levenshtein** - The number of characters that must be deleted, inserted, or substituted to make a pair of strings equal.

- **Jaccard** - The limit of overlay between 2 sets; in the case of Natural language processing (NLP), generally, documents are sets of words.

- **Smith Waterman** –Smith Waterman is identical to Levenshtein, but with costs appointed to substitution, insertion, and deletion.

Syntactic analysis also introduced as parsing, syntactic analysis is the function of analyzing strings as symbols and providing them according to a well-established set of grammatical rules. This step must, out of necessity, come before any further analysis, which attempts to extract insight from the text--semantic, sentiment, etc. -- treating it as something beyond symbols.



- **Tokenization:** Tokenization is the procedure of splitting up the given text into units called tokens.
- **Stopwords:** Stop words are words which are cleaned out before or after preparing natural language data (text).
- **Stemming:** A stemming is a procedure in which the various forms of similar words are reduced to a frequent form.
- **POS:** Tagging A POS Tagger is a sample of software that study text in some language and assign POS to each word, such as noun, verb, etc., despite usually computing operations use moreover fine-grained Part of speech tags like 'noun-plural'.

Semantic Analysis:

Semantic analysis is the phase in the compilation process that goes beyond syntax and focuses on the meaning of the source code. It aims to ensure that the program is not only syntactically correct but also semantically meaningful and adheres to the language's semantics rules. Semantic analysis checks for various aspects of program correctness, such as type checking, scoping rules, variable declarations, and the proper use of functions and data structures. It helps identify and report semantic errors, such as type mismatches or undeclared variables, which are not caught during the syntactic analysis. Semantic analysis plays a crucial role in ensuring the program's logic and behavior are consistent with the intended design and specification of the programming language.

- **Siamese Deep Neural Networks** for semantic similarity (SNN) A Siamese neural network (SNN) is an artificial neural network that uses equal pressure while working in tandem on two distinct input vectors to compute proportional output vectors. Generally, one of the output vectors is precomputed, thus constructing a baseline across which the other output vector is correlated. This is identical to match fingerprints but can be expressed more technically as a distance function for locality-sensitive hashing.

SNN is a class of neural network architectures that consists of two or more duplicate subnetworks. Duplicate here means they have a similar configuration with a similar framework and density. Framework renewing is illustrated over both subnetworks. Siamese NNs are famous among functions that require finding the same or a relation between two proportionate things. Some examples are paraphrased scoring, where the inputs are two sentences and the output is a score of how similar they are; or signature verification, where they figure out whether two signatures are from the same person. Generally, in such tasks, two identical subnetworks are used to process the two inputs, and another module will take their outputs and produce the final output.

- **Multi-head Attention Network Attention** takes two sentences, changes them into a matrix where the alteration of one sentence forms the columns, and the alterations of another sentence form the rows, and then it makes correlations, finding related context. This is especially helpful in machine translation. You don't just have to use attention to correspond meaning between sentences in two distinct languages. You can also put the similar sentence simultaneously the columns and the rows, to figure out how few parts of that sentence refer to others. So, study allows you to view at the completion of a sentence, to make relations between any specific word and its related context. This is highly diverse from the small-memory, upstream-focused RNNs, and quite specific from the proximity-focused convolutional networks.

Dataset

Our dataset releases will be oriented around various problems of relevance to Quora and will give researchers in diverse areas such as machine learning, natural language processing, network science, etc. the opportunity to try their hand at some of the challenges that arise in building a scalable online knowledge-sharing platform. Our first dataset is related to the problem of identifying duplicate questions.

The dataset that we are using will give anyone the opportunity to train and test models of semantic equivalence, based on actual Quora data.

Our dataset consists of over 400,000 lines of potential question duplicate pairs. Each line contains IDs for each question in the pair, the full text for each question, and a binary value that indicates whether the line truly contains a duplicate pair. Here are a few sample lines of the dataset:

id	qid1	qid2	question1	question2	is_duplicate
447	895	896	What are natural numbers?	What is a least natural number?	0
1518	3037	3038	Which pizzas are the most popularly ordered pizzas on Domino's menu?	How many calories does a Dominos pizza have?	0
3272	6542	6543	How do you start a bakery?	How can one start a bakery business?	1
3362	6722	6723	Should I learn python or Java first?	If I had to choose between learning Java and Python, what should I choose to learn first?	1

Here are a few important things to keep in mind about this dataset:

- Our original sampling method returned an imbalanced dataset with many more true examples of duplicate pairs than non-duplicates. Therefore, we supplemented the dataset with negative examples. One source of negative examples were pairs of “related questions” which, although pertaining to similar topics, are not truly semantically equivalent.
- The distribution of questions in the dataset should not be taken to be representative of the distribution of questions asked on Quora. This is, in part, because of the combination of sampling procedures and due to some sanitization measures, that have been applied to the final data set (e.g., removal of questions with extremely long question details).
- The ground-truth labels contain some amount of noise: they are not guaranteed to be perfect.

Given a set of two sentences, finding semantic similarity between those sentences is a difficult task as it requires analyzing a language which the system does not understand. (Human natural language). Paraphrase Detection has multiple real-world applications that solve real life problems as discussed earlier. We propose a system to detect paraphrasing of sentences in natural language using SNN in TensorFlow. By training the system in its initial phase via deep learning architecture such as CNN, RNN & MAN, preprocessing the sentence to extract features & derive results through a classifier in testing phase.

Implementation

System Requirements

Here are the system requirements for a paraphrase detection implementation project using BERT and TFIDF vectorizers:

Hardware Requirements:

- CPU: Intel Core i5 or higher
- RAM: 8GB or higher
- GPU: Nvidia GTX 1050 or higher (recommended for BERT fine-tuning)
- Storage: 500GB SSD for model and data files

Software Requirements:

- Operating System: Linux (Ubuntu 18.04 or later), Windows 10
- Programming Language: Python 3.6 or later
- Libraries/Frameworks: stremlit, sklearn, fuzzywuzzy, distance, bs4, scikit-learn.
- Pre-trained Models: BERT-base-uncased
- Other Software: Jupyter Notebook, TensorBoard

The hardware requirements assume running the models on a single machine. A more powerful GPU would be needed for distribution training of large datasets. The software requirements include the major Python packages for deep learning, NLP (BERT), vectorization (scikit-learn) and development (Jupyter). At minimum, an i5 CPU, 8GB RAM and GTX 1050 GPU should be sufficient to run the pretrained BERT and train/evaluate TFIDF and models on moderately sized datasets.

Bert Model

We introduce a language representation model called BERT, which stands for Bidirectional Encoder Representations from Transformers. Unlike recent language representation models, BERT is designed to pre-train deep bidirectional representations from unlabeled text by jointly conditioning on both left and right context in all layers. As a result, the pre-trained BERT model can be fine-tuned with just one additional output layer to create state-of-the-art models for a wide range of tasks, such as question answering and language inference, without substantial task-specific architecture modifications.

BERT is conceptually simple and empirically powerful. It obtains new state-of-the-art results on eleven natural language processing tasks, including pushing the GLUE score to 80.5% (7.7%-point absolute improvement), MultiNLI accuracy to 86.7% (4.6% absolute improvement), SQuAD v1.1 question answering Test F1 to 93.2 (1.5-point absolute improvement) and SQuAD v2.0 Test F1 to 83.1 (5.1-point absolute improvement).

BERT is an open source machine learning framework for natural language processing (NLP). BERT is designed to help computers understand the meaning of ambiguous language in text by using surrounding text to establish context. The BERT framework was pre-trained using text from Wikipedia and can be fine-tuned with question-and-answer datasets.

BERT, which stands for Bidirectional Encoder Representations from Transformers, is based on Transformers, a deep learning model in which every output element is connected to every input element, and the weightings between them are dynamically calculated based upon their connection. (In NLP, this process is called attention.)

Historically, language models could only read text input sequentially -- either left-to-right or right-to-left -- but couldn't do both at the same time. BERT is different because it is designed to read in both directions at once. This capability, enabled by the introduction of Transformers, is known as bidirectionality.

Using this bidirectional capability, BERT is pre-trained on two different, but related, NLP tasks: Masked Language Modeling and Next Sentence Prediction.

The objective of Masked Language Model (MLM) training is to hide a word in a sentence and then have the program predict what word has been hidden (masked) based on the hidden word's context. The objective of Next Sentence Prediction training is to have the program predict whether two given sentences have a logical, sequential connection or whether their relationship is simply random.

TF-IDF Vectorizers

TF-IDF approach was a state-of-the-art method for many years before the advent of Word Embeddings for textual knowledge representation for Machine Learning algorithms. Thus, to compare our approach with two baselines that do not employ Deep Learning technologies, we decided to use TF-IDF and SVM model and TF-IDF and Multilayer Perceptron model.

Using Streamlit

This code is a Python script that uses the Streamlit library to create a simple web application for paraphrase detection. It loads a pre-trained machine learning model (presumably for paraphrase detection) from a pickled file, takes two input text questions, and then predicts whether the input questions are paraphrased or not.

Here's a breakdown of the code:

1. Import necessary libraries:
 - streamlit: This library is used for creating web applications with minimal effort.
 - helper: It's assumed that there is a Python module called helper containing the query_point_creator function.
 - pickle: This library is used to deserialize (unpickle) the pre-trained machine learning model stored in a file.
2. Load the pre-trained machine learning model:
 - The pickle.load function is used to load a pickled model file named 'model.pkl' with read-binary ('rb') mode. This model is presumably a trained model for paraphrase detection.
3. Create a Streamlit web application:
 - The Streamlit app is created with a header labeled 'Paraphrase Detection'.
4. Collect user input:
 - Two text input fields, q1 and q2, are provided for users to enter two questions they want to compare for paraphrasing.
5. Check for a button press:
 - The code uses st.button to create a button labeled 'Find'. This button allows the user to trigger the paraphrase detection process.
6. Perform paraphrase detection:
 - When the 'Find' button is clicked, the code calls the query_point_creator function from the 'helper' module,

```
import streamlit as st
import helper
import pickle

model = pickle.load(open('model.pkl','rb'))

st.header('Paraphrase Detection')

q1 = st.text_input('Enter question 1')
q2 = st.text_input('Enter question 2')

if st.button('Find'):
    query = helper.query_point_creator(q1,q2)
    result = model.predict(query)[0]

    if result:
        st.header('The Sentences Are Paraphrased.')
    else:
        st.header('The Sentences Are Not Paraphrased.')
```

Feature Extraction and Data Preprocessing

In the code we provided earlier, which is a part of the `query_point_creator` function, BERT machine learning algorithm is used. And the code is focused on **feature extraction and data preprocessing**. It calculates various features from the input text and generates feature vectors, which can then be used as input data for machine learning algorithms of your choice.

The code involves the following operations:

- Preprocessing of input text (q1 and q2) to clean and transform the text.
- Calculation of various features, including common word count, common stopword count, length-based features, fuzzy string-matching features, and Bag-of-Words (BoW) representations of the text using a Count Vectorizer (cv).
- These calculated features are then combined into a single feature vector.

It's up to you to decide which machine learning algorithm to use with this feature vector, depending on the specific NLP task you want to perform. The code prepares the data by extracting features, and the actual machine learning algorithm would be applied separately in your NLP project based on your specific task, such as text classification, similarity measurement, or other NLP-related tasks.

This code appears to be part of a Natural Language Processing (NLP) project that aims to build a model for comparing and measuring the similarity between two text-based questions or sentences. The code performs a series of preprocessing, feature extraction, and data transformation steps to create a set of features for comparing two questions.

Here's a breakdown of the code:

Imports: The code starts by importing several libraries and modules used in the subsequent functions, including regular expressions (re), BeautifulSoup for HTML parsing, distance for string similarity calculations, fuzzywuzzy for fuzzy string matching, pickle for loading pre-trained models, and numpy for numerical operations.

Loading a Count Vectorizer: It loads a Count Vectorizer (cv) from a pickle file called 'cv.pkl.' A Count Vectorizer is a method for converting a collection of text documents to a matrix of token counts.

Utility Functions:

- `test_common_words(q1, q2)`: This function takes two strings, splits them into words, and returns the count of common words between the two strings.
- `test_total_words(q1, q2)`: This function returns the total number of words in both input strings.

- `test_fetch_token_features(q1, q2)`: This function calculates various token-based features such as common word count, common stop word count, and common token count between two input strings.
- `test_fetch_length_features(q1, q2)`: This function computes various length-based features, including the absolute length difference, average token length, and the length of the longest common substring between two strings.
- `test_fetch_fuzzy_features(q1, q2)`: This function uses fuzzy string-matching methods to calculate four features: fuzz ratio, fuzz partial ratio, token sort ratio, and token set ratio.

Preprocessing Function `preprocess(q)`:

This function takes a string `q` as input and performs a series of text preprocessing steps, including lowercasing, replacing special characters, handling numbers, decontracting words, removing HTML tags, and removing punctuation. It returns the preprocessed string.

`query_point_creator(q1, q2)` Function:

- This function takes two input strings, `q1` and `q2`.
- It first preprocesses both strings using the `preprocess` function.
- It calculates some basic features, such as the length of each question and the number of words in each question.
- It then calculates common word and token-based features using the `test_common_words` and `test_total_words` functions.
- It also calculates features based on the tokenized words of the input strings using the `test_fetch_token_features` function.
- Features related to the length of the input strings are computed using the `test_fetch_length_features` function.
- Fuzzy string-matching features are calculated using the `test_fetch_fuzzy_features` function.
- Finally, it creates Bag-of-Words (BoW) representations of both `q1` and `q2` using the Count Vectorizer (`cv`).

The function returns a NumPy array containing all the calculated features, which can be used as input data for a machine learning model to predict the similarity between the two input questions.

"Fuzz ratio," "fuzz partial ratio," "token sort ratio," and "token set ratio" are string similarity measures often used in natural language processing and text analysis to compare the similarity between two strings. They are particularly useful when dealing with approximate string matching or comparing strings that may have minor differences. Here's an explanation of each:

Fuzz Ratio:

Fuzz ratio often calculated using the `fuzz.QRatio` function, measures the similarity between two strings as a ratio. It compares the two strings character by character and calculates a similarity score as a percentage. The higher the ratio, the more similar the strings are.

Formula: $(2 * \text{Number of matching characters}) / (\text{Length of the first string} + \text{Length of the second string})$

Example: If you have "hello" and "helo," the fuzz ratio might be high because the strings are similar.

Fuzz Partial Ratio:

Fuzz partial ratio, calculated using the `fuzz.partial_ratio` function, is like fuzz ratio but more forgiving. It looks for the best partial match between two strings and calculates the similarity score based on that partial match.

Example: If you have "hello" and "hel," the fuzz partial ratio might be high because "hel" is a substring of "hello."

Token Sort Ratio:

Token sort ratio, calculated using the `fuzz.token_sort_ratio` function, is used for comparing strings where the words or tokens are in a different order. It first sorts the words or tokens in both strings and then calculates the fuzz ratio on the sorted versions.

Example: If you have "apple banana cherry" and "cherry apple banana," the token sort ratio would be high because the content is the same, even though the word order differs.

Token Set Ratio:

Token set ratio, calculated using the `fuzz.token_set_ratio` function, is useful for comparing strings where there may be some words in common but not necessarily in the same order. It tokenizes the strings, calculates the ratio of common tokens, and considers token set operations like intersection and union.

Example: If you have "I like apples" and "Apples are my favorite," the token set ratio might be high because both strings share common tokens like "apples."

These similarity measures are valuable in various applications, such as information retrieval, search engines, spell-checkers, and fuzzy string-matching tasks where the exact string matching may not be practical or necessary. They provide a way to quantify the degree of similarity between two strings, making them useful in tasks like duplicate detection, record linkage, and recommendation systems.

Testing and Validation

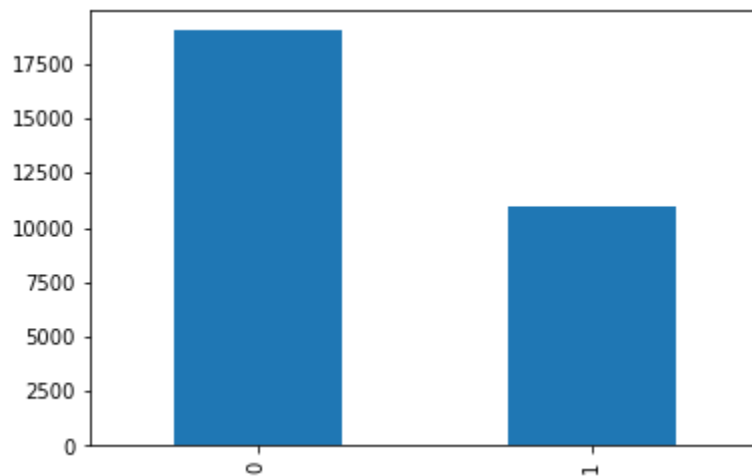
We have used test.csv from Quora-Questions-Pair to perform various checkpoints for processing the data.

Distribution of duplicate and non-duplicate questions

```
print(new_df['is_duplicate'].value_counts())
```

```
print((new_df['is_duplicate'].value_counts()/new_df['is_duplicate'].count()*100)
```

```
new_df['is_duplicate'].value_counts().plot(kind='bar')
```



0 is dis-similar text.

1 is similar text.

Repeated questions

```
qid = pd.Series(new_df['qid1'].tolist() + new_df['qid2'].tolist())
```

```
print('Number of unique questions',np.unique(qid).shape[0])
```

```
x = qid.value_counts()>1
```

```
print('Number of questions getting repeated',x[x].shape[0])
```

Number of unique questions 55299

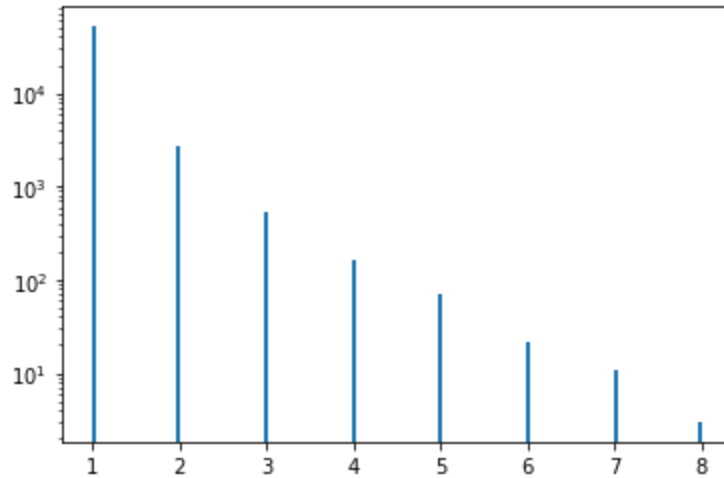
Number of questions getting repeated 3480

Repeated questions histogram

```
plt.hist(qid.value_counts().values,bins=160)
```

```
plt.yscale('log')
```

```
plt.show()
```



The range of question counts will be divided into 160 bins, and the histogram will show the distribution of question counts.

The resulting plot will show a histogram of question counts, where the x-axis represents the number of times a question is repeated, and the y-axis represents the frequency (in a logarithmic scale) of questions falling into each count category. This can help you visualize how many questions are repeated a certain number of times.

Feature Engineering

```
new_df['q1_len'] = new_df['question1'].str.len()
```

```
new_df['q2_len'] = new_df['question2'].str.len()
```

```
new_df.head()
```

	id	qid1	qid2	question1	question2	is_duplicate	q1_len	q2_len
398782	398782	496695	532029	What is the best marketing automation tool for...	What is the best marketing automation tool for...	1	76	77
115086	115086	187729	187730	I am poor but I want to invest. What should I do?	I am quite poor and I want to be very rich. Wh...	0	49	57
327711	327711	454161	454162	I am from India and live abroad. I met a guy f...	T.I.E.T to Thapar University to Thapar Univers...	0	105	120
367788	367788	498109	491396	Why do so many people in the U.S. hate the sou...	My boyfriend doesnt feel guilty when he hurts ...	0	59	146
151235	151235	237843	50930	Consequences of Bhopal gas tragedy?	What was the reason behind the Bhopal gas trag...	0	35	50

Number of Words

```
new_df['q1_num_words'] = new_df['question1'].apply(lambda row: len(row.split(" ")))  
new_df['q2_num_words'] = new_df['question2'].apply(lambda row: len(row.split(" ")))  
new_df.head()
```

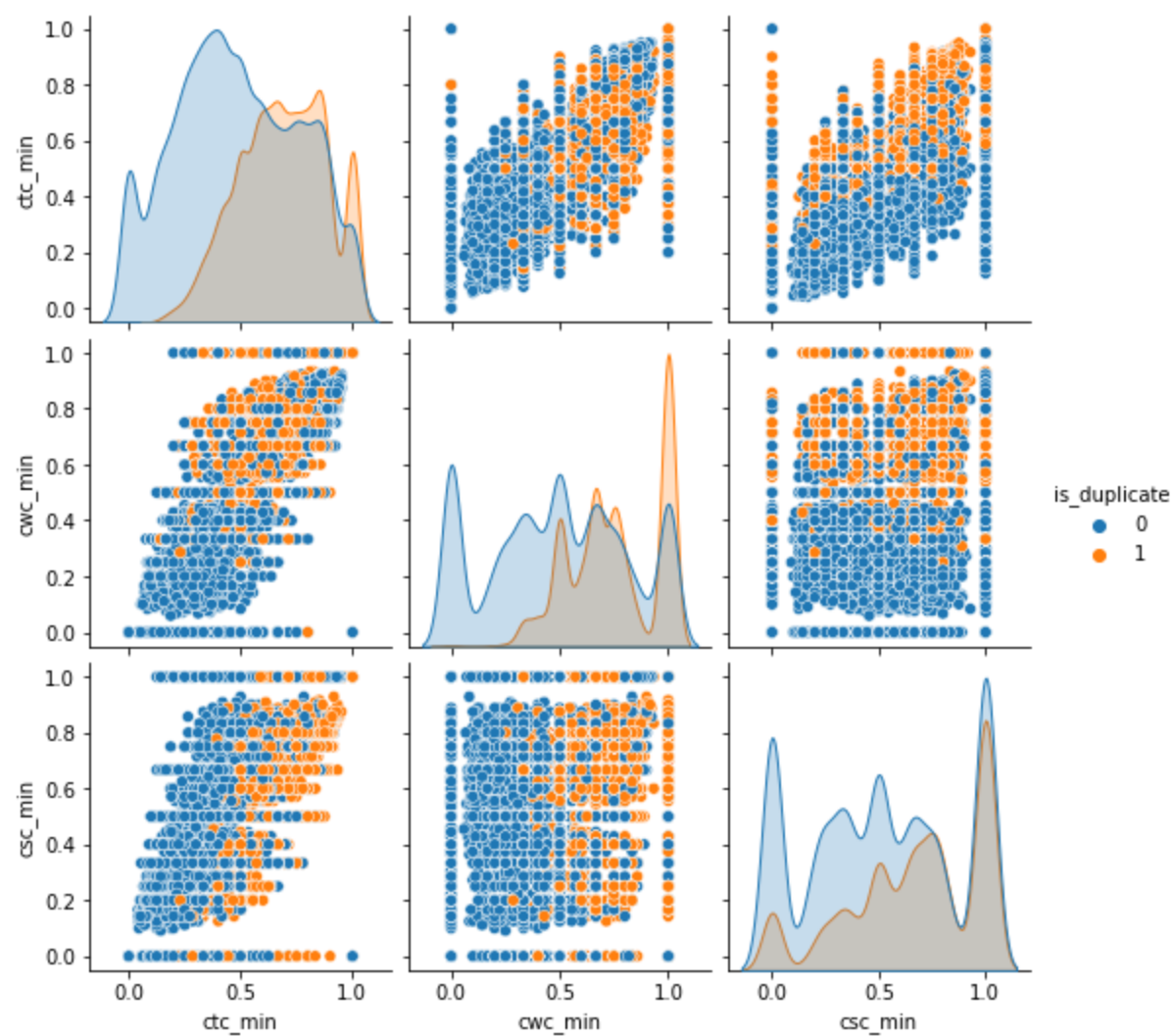
	id	qid1	qid2	question1	question2	is_duplicate	q1_len	q2_len	q1_num_words	q2_num_words
398782	398782	496695	532029	What is the best marketing automation tool for...	What is the best marketing automation tool for...	1	76	77	12	12
115086	115086	187729	187730	I am poor but I want to invest. What should I do?	I am quite poor and I want to be very rich. Wh...	0	49	57	12	15
327711	327711	454161	454162	I am from India and live abroad. I met a guy f...	T.I.E.T to Thapar University to Thapar Univers...	0	105	120	25	17
367788	367788	498109	491396	Why do so many people in the U.S. hate the sou...	My boyfriend doesnt feel guilty when he hurts ...	0	59	146	12	30
151235	151235	237843	50930	Consequences of Bhopal gas tragedy?	What was the reason behind the Bhopal gas trag...	0	35	50	5	9

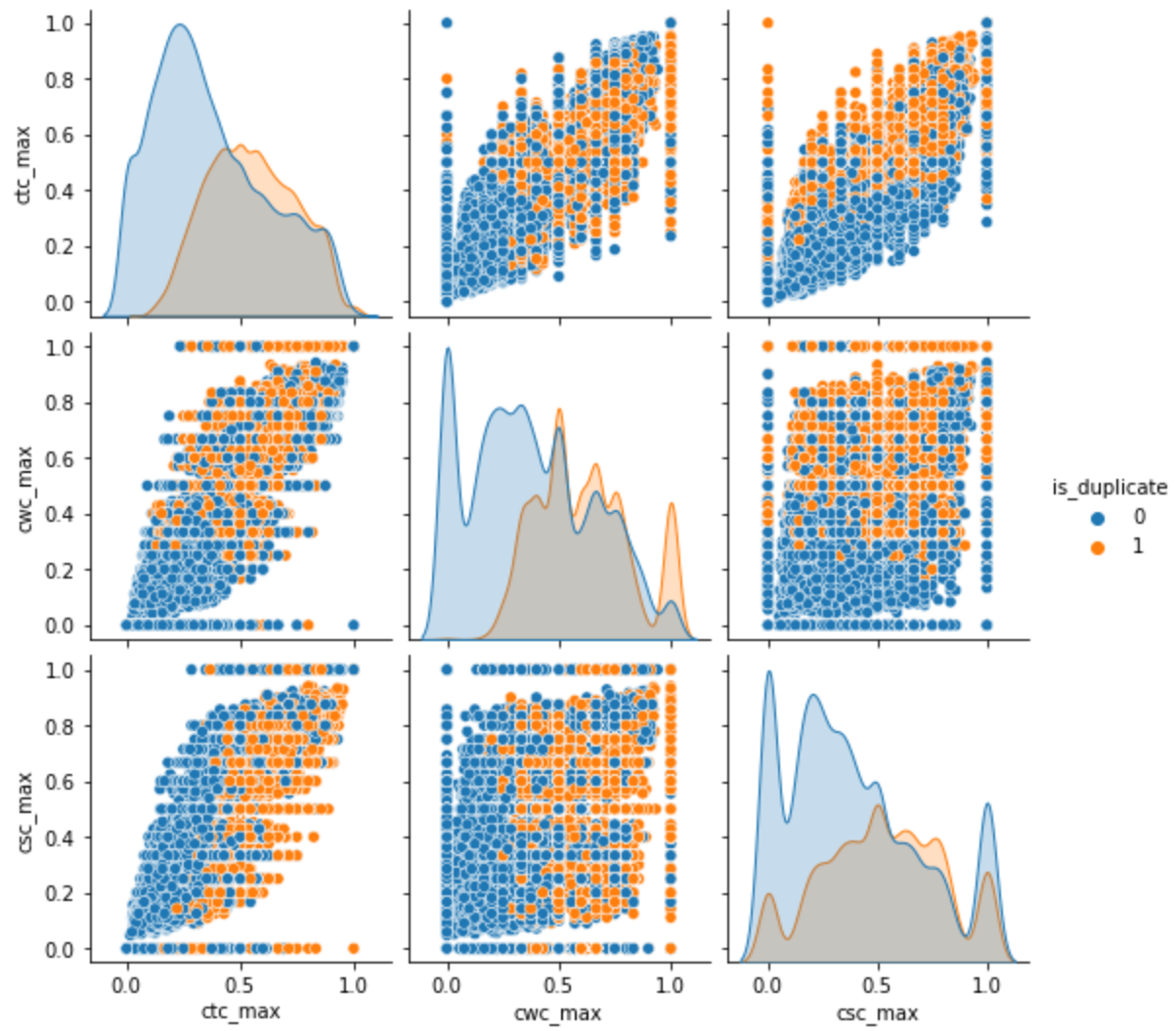
Preprocessing and Advance Features Checkpoint

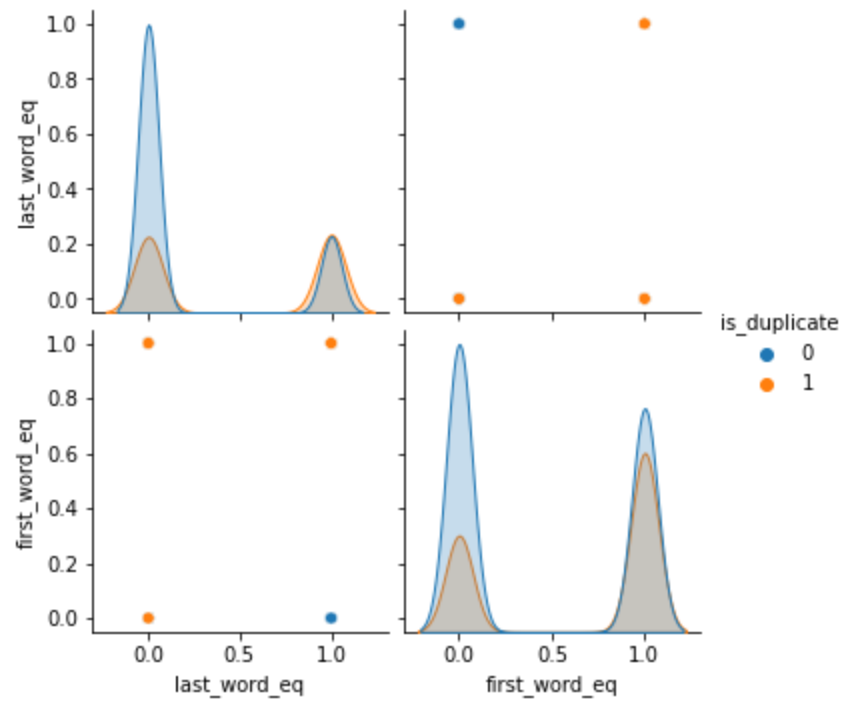
Advanced Features

1. Token Features

- **cwc_min**: This is the ratio of the number of common words to the length of the smaller question.
- **cwc_max**: This is the ratio of the number of common words to the length of the larger question.
- **csc_min**: This is the ratio of the number of common stop words to the smaller stop word count among the two questions **csc_max**: This is the ratio of the number of common stop words to the larger stop word count among the two questions.
- **ctc_min**: This is the ratio of the number of common tokens to the smaller token count among the two questions.
- **ctc_max**: This is the ratio of the number of common tokens to the larger token count among the two questions.
- **last_word_eq**: 1 if the last word in the two questions is same, 0 otherwise.
- **first_word_eq**: 1 if the first word in the two questions is same, 0 otherwise.

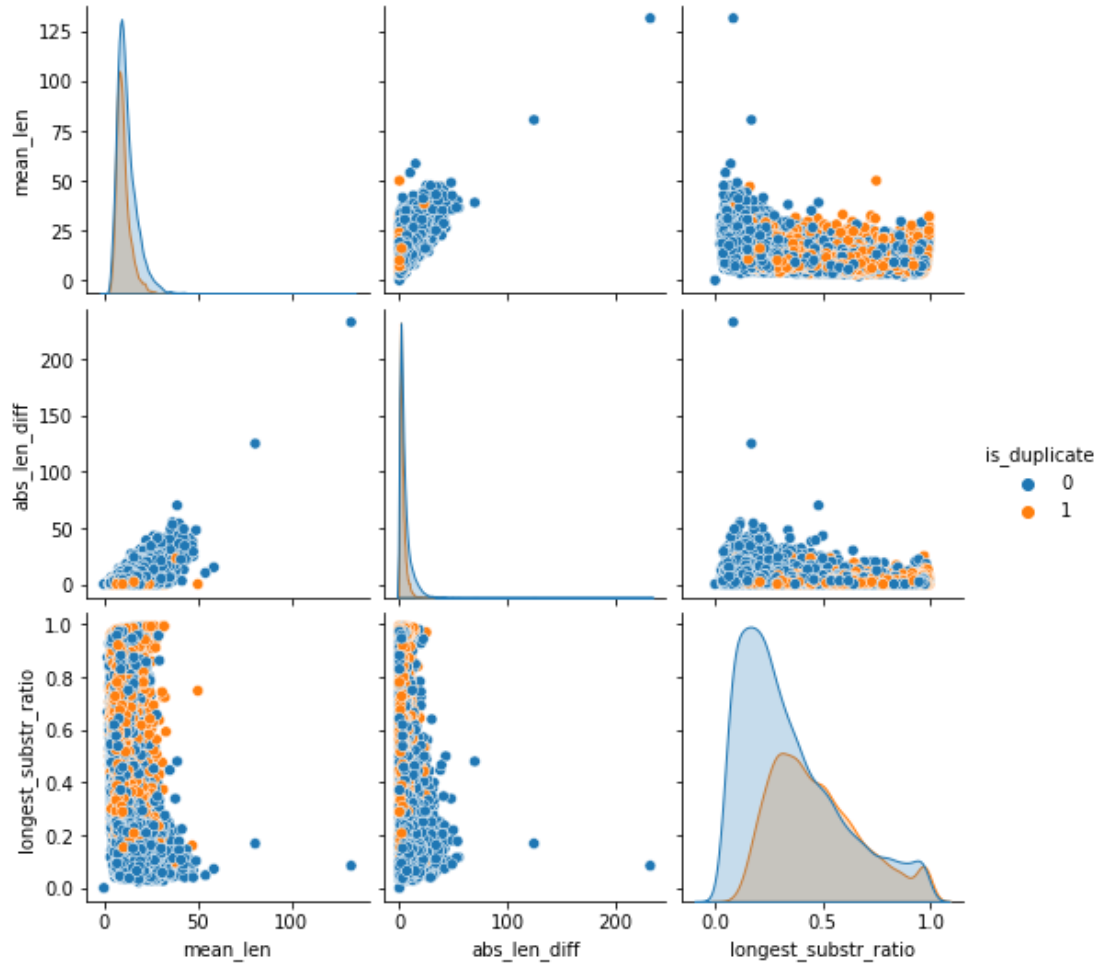






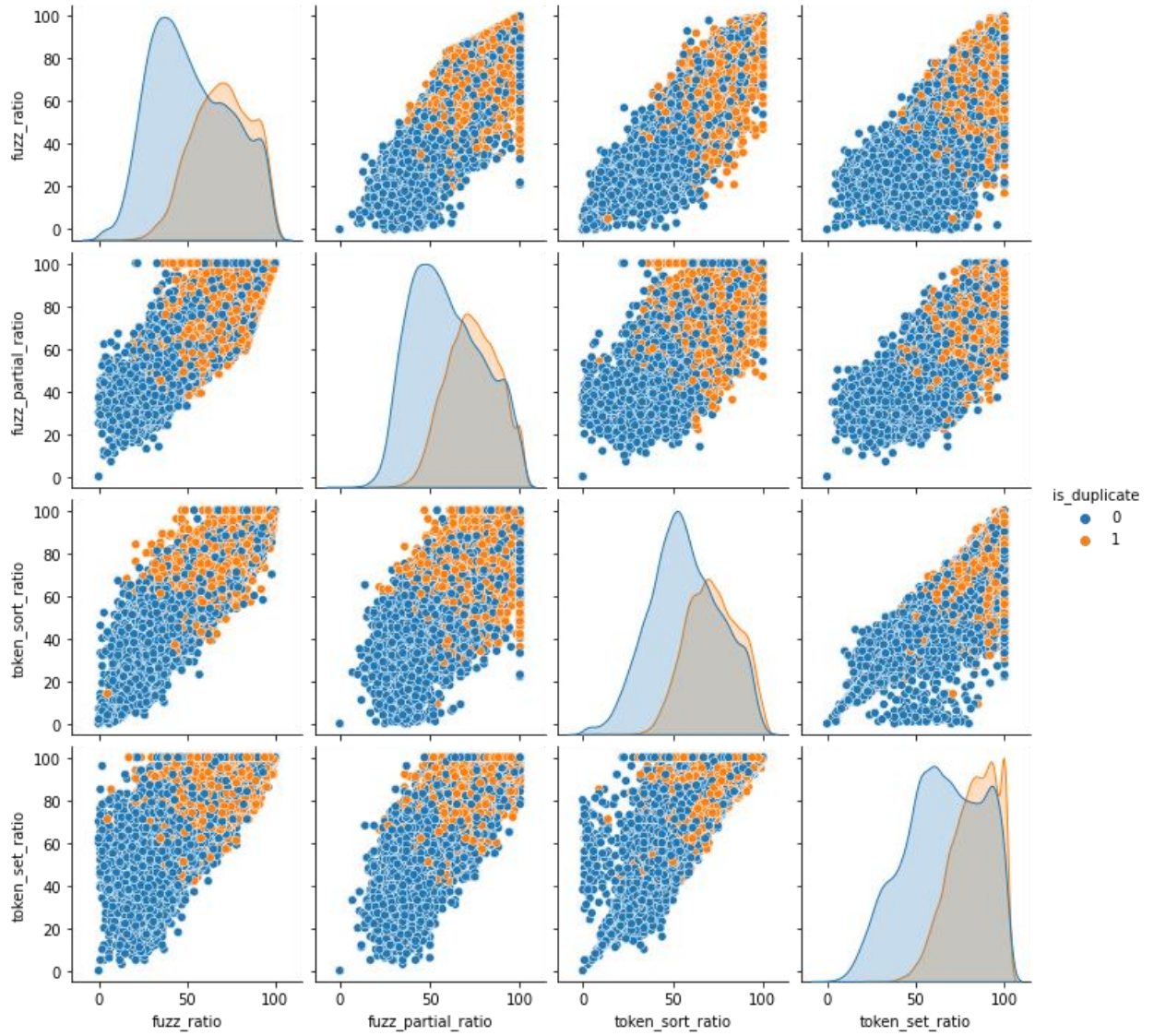
2. Length Based Features

- mean_len: Mean of the length of the two questions (number of words)
- abs_len_diff. Absolute difference between the length of the two questions (number of words)
- longest_substr_ratio: Ratio of the length of the longest substring among the two questions to the length of the smaller question



3. Fuzzy Features

- fuzz_ratio: fuzz ratio score from fuzzywuzzy
- fuzz_partial_ratio: fuzz_partial_ratio from fuzzywuzzy
- token_sort_ratio: token_sort_ratio from fuzzywuzzy
- token_set_ratio: token_set_ratio from fuzzywuzzy



The code you provided uses the CountVectorizer from scikit-learn (`sklearn.feature_extraction.text`) to convert text data into a bag of words representation. It appears to be processing two columns of text data from a Data Frame `ques_df`, namely 'question1' and 'question2', and then splitting the resulting bag of words representations into two arrays `q1_arr` and `q2_arr`. Here's a breakdown of what the code does:

The code provided uses the `train_test_split` function from the `sklearn.model_selection` module to split your data into training and testing sets.

- `temp_df` is assumed to be a Data Frame.
- `temp_df.iloc[:, 0:-1]` selects all the rows and all columns of the Data Frame except for the last column. This is assumed to be your feature data, often denoted as `X`.
- `temp_df.iloc[:, -1]` selects all the rows and the last column of the Data Frame. This is assumed to be your target or label data, often denoted as `y`.
- `train_test_split` is used to split the feature data (`X`) and target data (`y`) into training and testing sets.
- `X_train` will contain the training feature data.
- `X_test` will contain the testing feature data.
- `y_train` will contain the training target data.
- `y_test` will contain the testing target data.

The `test_size` parameter is set to 0.2, which means that 20% of the data will be used for testing, and the remaining 80% will be used for training.

The `random_state` parameter is set to 1, which ensures that the data split is reproducible. If you use the same `random_state` value in the future, you'll get the same data split.

After running this code, you will have your data split into training and testing sets, allowing you to train a machine learning model on the training data and evaluate its performance on the testing data.

After executing this code, you will have `q1_arr` and `q2_arr`, which contain bag-of-words representations of the text in 'question1' and 'question2', respectively. These representations can be used as input features for various machine learning models.

On our training data and then evaluate its performance on your testing data. Here's a breakdown of the code:

1. Import the necessary modules from scikit-learn:
 - a. RandomForestClassifier from sklearn.ensemble is imported for building a random forest classifier.
 - b. accuracy_score from sklearn.metrics is imported to calculate the accuracy of the classifier.
2. Create an instance of the RandomForestClassifier and store it in the variable rf. This creates a random forest classifier with default hyperparameters.
3. Train the classifier using your training data:
 - a. rf.fit(X_train, y_train) fits the random forest classifier to the training data, where X_train is the training feature data, and y_train is the corresponding target labels.
4. Make predictions using the trained classifier on your testing data:
 - a. y_pred = rf.predict(X_test) uses the trained random forest classifier to predict the labels for the testing feature data X_test. The predictions are stored in the y_pred variable.
5. Calculate the accuracy of the classifier:
 - a. accuracy_score(y_test, y_pred) compares the predicted labels (y_pred) with the actual labels (y_test) from the testing data and calculates the accuracy of the model.

After running this code, the accuracy_score will give you the accuracy of the random forest classifier on your testing data, which is a measure of how well the classifier performs in terms of correctly classifying the data. Higher accuracy indicates better performance.

Confusion Matrix

The `confusion_matrix` function is used to generate a confusion matrix, which is a table that is often used to evaluate the performance of a classification model. It displays the true positive, true negative, false positive, and false negative counts for a classification problem.

In this code, `y_test` should be the true labels from your testing dataset, and `y_pred` should be the predicted labels generated by your classification model. The `confusion_matrix` function will return a 2x2 matrix (in a binary classification problem) or a larger matrix (in a multi-class problem) with the following layout:

	Predicted Negative	Predicted Positive
Actual Negative	TN	FP
Actual Positive	FN	TP

- TN (True Negative): The number of samples that were correctly predicted as negative (correctly classified as the negative class).
- FP (False Positive): The number of samples that were incorrectly predicted as positive (misclassified as the positive class when they were actually negative).
- FN (False Negative): The number of samples that were incorrectly predicted as negative (misclassified as the negative class when they were actually positive).
- TP (True Positive): The number of samples that were correctly predicted as positive (correctly classified as the positive class).

The confusion matrix helps you assess the performance of your classification model and is often used to calculate various performance metrics such as accuracy, precision, recall, and F1-score.

Performance metrics are important for evaluating the effectiveness of a classification model. Here are some key performance metrics and their definitions:

1. Accuracy:

- Accuracy is a measure of the overall correctness of a model.
- It is calculated as $(TP + TN) / (TP + TN + FP + FN)$, where:
- TP (True Positives) is the number of correctly predicted positive instances.
- TN (True Negatives) is the number of correctly predicted negative instances.

- FP (False Positives) is the number of negative instances incorrectly predicted as positive.
- FN (False Negatives) is the number of positive instances incorrectly predicted as negative.

2. Precision:

- Precision is a measure of the accuracy of positive predictions made by the model.
- It is calculated as $TP / (TP + FP)$, where TP is the number of true positives and FP is the number of false positives.
- Precision answers the question: "Of all the positive predictions, how many were correct?"

3. Recall (Sensitivity or True Positive Rate):

- Recall is a measure of how many of the actual positive instances were correctly predicted as positive by the model.
- It is calculated as $TP / (TP + FN)$, where TP is the number of true positives and FN is the number of false negatives.
- Recall answers the question: "Of all the actual positives, how many did the model correctly identify?"

4. F1-Score:

- The F1-Score is the harmonic mean of precision and recall.
- It balances precision and recall and provides a single metric that considers both false positives and false negatives.
- It is calculated as $2 * (Precision * Recall) / (Precision + Recall)$.

These metrics are widely used to evaluate classification models. The choice of which metric to prioritize depends on the specific problem and the trade-off between false positives and false negatives. In some cases, precision might be more critical (e.g., in medical diagnoses), while in others, recall might be more important (e.g., spam email detection). The F1-Score is often used when you want a balance between precision and recall.

We are using the pickle module to save your trained RandomForestClassifier (rf) and CountVectorizer (cv) objects to disk. This allows you to store the trained model and vectorizer for later use without the need to retrain or recompute them. Here's a breakdown of what the code does:

`pickle.dump(rf, open('model.pkl', 'wb'))`: This line saves your RandomForestClassifier rf to a file named 'model.pkl' in binary write mode ('wb'). The model is serialized and saved as a binary file.

`pickle.dump(cv, open('cv.pkl', 'wb'))`: This line saves your CountVectorizer cv to a file named 'cv.pkl' in binary write mode. The CountVectorizer is also serialized and saved as a binary file.

After executing these lines of code, you will have two files, 'model.pkl' and 'cv.pkl', which store your trained model and CountVectorizer. You can later load these objects back into your Python environment using the `pickle.load` function to make predictions or use the vectorizer with new text data without having to retrain them.

Result

This code takes your two example sentences, creates a feature vector using the `query_point_creator` function, and then uses your pre-trained model to predict whether the sentences are paraphrased or not. The result will be printed based on the prediction made by your model.

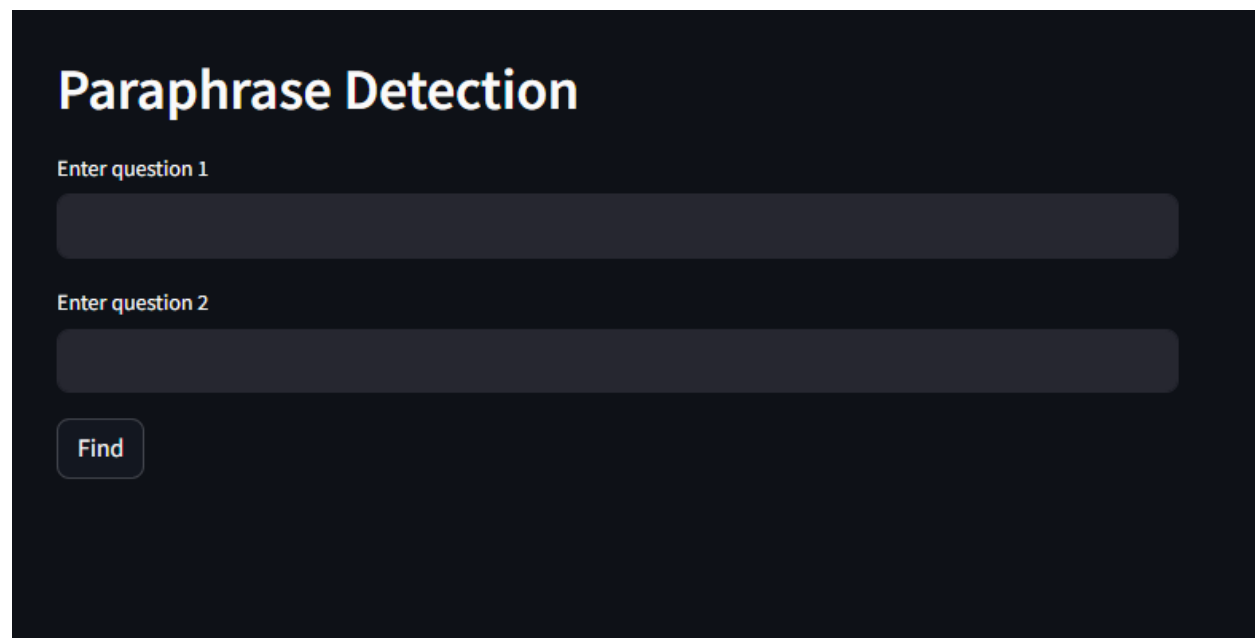
Make sure that 'model.pkl' contains a valid, pre-trained model that you intend to use for paraphrase detection. Additionally, ensure that your helper module contains the `query_point_creator` function and any other necessary functions.

Before running the command, we must make sure that all the requirements are installed.

Run Command:

```
C:\Windows\System32\cmd.e  X  +  v
Microsoft Windows [Version 10.0.22621.2428]
(c) Microsoft Corporation. All rights reserved.

D:\Home\Dennis\Project\quora-question-pairs\quora-question-pairs\streamlit-app>streamlit run app.py|
```



Paraphrase Detection

Enter question 1

Enter question 2

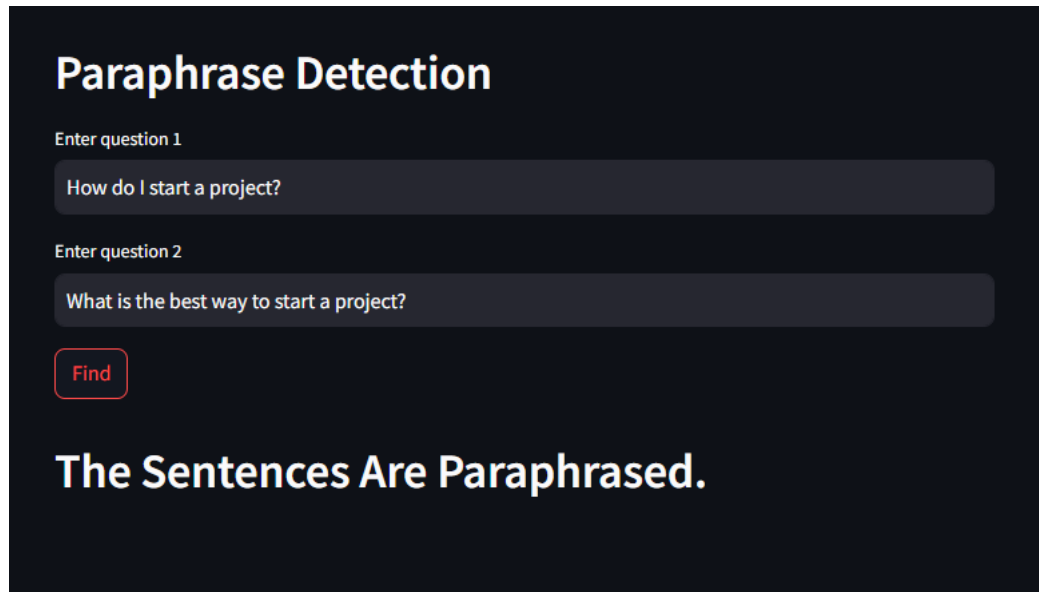
Find

Input: For Positive Test Case

Q1: How do I start a project?

Q2: What is the best way to start a project?

Here even the sentences seem completely different, but they carry the same meaning.



The screenshot shows a dark-themed web interface titled "Paraphrase Detection". It has two input fields labeled "Enter question 1" and "Enter question 2". The first field contains the text "How do I start a project?" and the second field contains "What is the best way to start a project?". Below these fields is a red-outlined button labeled "Find". At the bottom of the interface, a large white text message states "The Sentences Are Paraphrased."

Input: For Negative Test Case

Q1: How do you waste time?

Q2: How do I write an essay?

Here the sentences look almost the same, but they carry a different meaning.



The screenshot shows a dark-themed web interface titled "Paraphrase Detection". It has two input fields labeled "Enter question 1" and "Enter question 2". The first field contains the text "How do I start a project?" and the second field contains "How do I write an essay?". Below these fields is a red-outlined button labeled "Find". At the bottom of the interface, a large white text message states "The Sentences Are Not Paraphrased."

Conclusion

In summary, our code sets up a simple web application using Streamlit that allows users to input two questions or sentences. It then uses a pre-trained machine learning model to predict whether the input sentences are paraphrased or not and displays the result on the web page. The 'helper' module is likely used to preprocess and format the input data for the model.

In this work, we experimented Bag of Words with boosting algorithms, Catboost, and XGBoost. We also tested Quora Question Pairs using BERT. The results show that BERT gave the best result among other models. For further research, we suggest that more work on paraphrase generation which may be more significant to the natural language processing task such as machine translation and question answering system.

For future research, we would pursue pre-training on other forms of self-supervised objectives, and perhaps use contrastive learning to improve pre-training. We also found that a multiple-layer neural network for task-specific heads yields better results than a single naive linear layer for each task-head module. In the future, weight sharing between some of these task-specific modules may improve parameter efficiency and lead to more generalized learning across tasks. BERT is known to do better on larger datasets. We noticed significant over-fitting for the sentiment analysis and semantic text similarity tasks. In our future efforts, implementing data augmentation would be a good way to lower over-fitting and help the model generalize better. Based on the results, the model is doing better in capturing semantic similarity between sentences than in qualifying the emotion of the sentence. It might also help to experiment with other neural network architectures for the sentiment analysis head to improve performance for classifying sentiment.

References

- ChiSun,XipengQiu,YigeXu,andXuanjingHuang.Howtofine-tuneBERTfortextclassification?ChineseComputationalLinguistics,2019.
- JacobDevlin,Ming-WeiChang,KentonLee,andKristinaToutanova.Bert:Pre-trainingof deepbidirectionaltransformersforlanguageunderstanding.ArXiv,abs/1810.04805,2019.
- Mohri, M., Rostamizadeh, A., & Talwalkar, A. (2012). Foundations of machine learning. MIT press.
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. nature, 521(7553), 436. International Journal of Computer Applications (0975 – 8887) Volume 182 – No. 46, March 2019 5
- Mihalcea, R., Corley, C., & Strapparava, C. (2006, July). Corpus-based and knowledge-based measures of text semantic similarity. In AAAI (Vol. 6, pp. 775-780).
- Hassan, S. (2011). Measuring semantic relatedness using salient encyclopedic concepts. University of North Texas.
- Rus, V., McCarthy, P. M., Lintean, M. C., McNamara, D. S., & Graesser, A. C. (2008, May). Paraphrase Identification with Lexico-Syntactic Graph Subsumption. In FLAIRS conference(pp. 201-206).
- Islam, A., & Inkpen, D. (2009). Semantic similarity of short texts. Recent Advances in Natural Language Processing V, 309, 227-236.
- Milajevs, D., Kartsaklis, D., Sadrzadeh, M., & Purver, M. (2014). Evaluating neural word representations in tensor-based compositional settings. arXiv preprint arXiv:1408.6179.
- Fernando, S., & Stevenson, M. (2008, March). A semantic similarity approach to paraphrase detection. In Proceedings of the 11th Annual Research Colloquium of the UK Special Interest Group for Computational Linguistics (pp. 45-52).
- Qiu, L., Kan, M. Y., & Chua, T. S. (2006, July). Paraphrase recognition via dissimilarity significance classification. In Proceedings of the 2006 Conference on Empirical Methods in Natural Language Processing (pp. 18-26). Association for Computational Linguistics.
- Ul-Qayyum, Z., & Altaf, W. (2012). Paraphrase identification using semantic heuristic

features. *Research Journal of Applied Sciences, Engineering and Technology*, 4(22), 4894-4904.

- Kozareva, Z., & Montoyo, A. (2006). Paraphrase identification on the basis of supervised machine learning techniques. In *Advances in natural language processing* (pp. 524-533). Springer, Berlin, Heidelberg.
- Ji, Y., & Eisenstein, J. (2013). Discriminative improvements to distributional sentence similarity. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing* (pp. 891-896).
- Filice, S., Da San Martino, G., & Moschitti, A. (2015). Structural representations for learning relations between pairs of texts. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)* (Vol. 1, pp. 1003-1013).
- Socher, R., Huang, E. H., Pennin, J., Manning, C. D., & Ng, A. Y. (2011). Dynamic pooling and unfolding recursive auto-encoders for paraphrase detection. In *Advances in neural information processing systems* (pp. 801-809).
- Blacoe, W., & Lapata, M. (2012, July). A comparison of vector-based representations for semantic composition. In *Proceedings of the 2012 joint conference on empirical methods in natural language processing and computational natural language learning* (pp. 546-556). Association for Computational Linguistics.
- He, H., Gimpel, K., & Lin, J. (2015). Multi-perspective sentence similarity modeling with convolutional neural networks. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing* (pp. 1576-1586).
- Cheng, J., & Kartsaklis, D. (2015). Syntax-aware multi-sense word embedding for deep compositional models of meaning. *arXiv preprint arXiv:1508.02354*.
- Wang, Z., Mi, H., & Ittycheriah, A. (2016). Sentence similarity learning by lexical decomposition and composition. *arXiv preprint arXiv:1602.07019*
- H. Schmid, Probabilistic part-of-speech tagging using decision trees, in: *Proceedings of International Conference on New Methods in Language Processing*, pp. 44–49, Manchester, September 1994.
- M. Potthast, B. Stein, A. Eiselt, A. Barrón-Cedeño and P. Rosso, Overview of the 2nd International competition on plagiarism detection, in: *Notebook Papers of CLEF 2010 Labs*

and Workshops, M. Braschler, and D. Harman, eds., Italy, September 2010.

- R. Bhagat and E. Hovy, “Squibs: What is a paraphrase?” *Computational Linguistics*, vol. 39, no. 3, pp. 463–472, Sep. 2013. [Online]. Available: <https://aclanthology.org/J13-3001>
- A. Fujita and P. Isabelle, “Expanding paraphrase lexicons by exploiting generalities,” *ACM Transactions on Asian and Low-Resource Language Information Processing*, vol. 17, no. 2, pp. 1–36, 2018. [Online]. Available: <https://dl.acm.org/doi/10.1145/3160488>
- M. AL-Smadi, Z. Jaradat, M. AL-Ayyoub, and Y. Jararweh, “Paraphrase identification and semantic text similarity analysis in arabic news tweets using lexical, syntactic, and semantic features,” *Information Processing and Management: an International Journal*, vol. 53, no. 3, pp. 640–652, 2017. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0306457316302382>